

Project 1: Atomic Environment Classification as a Tool for Crystal Growth and Phase Transition Visualization

[Unsupervised Learning, Molecular Descriptors]

Reference: João Paulo Almeida de Mendonça

Objective: You will use machine learning techniques to classify atomic environments in molecular dynamics (MD) trajectories and visualize crystal growth and phase transitions, trying to get a better visualization approach that can identify phases more precisely than traditional metrics like coordination number.

Background: In simulations of crystallization or solid-solid transitions, local atomic environments shift significantly. Traditional descriptors like [coordination number](#), [effective coordination number \(ECN\)](#) or [common neighbor analysis \(CNA\)](#) offer some insight but may not be the best for every system. By applying molecular descriptors and clustering/classification techniques, you can identify and visualize distinct atomic environments automatically.

Tools & Libraries:

- Everything can be done in a Notebook, using !pip to install packages.
- ASE or [Ovito](#) are helpful to manipulate the structures and read the MD files.
- Descriptors: Coulomb Matrix, SOAP, ACSF... All are available via [DScribe](#), or you can use any manual feature engineering/code you want.
- CN, ECN, and CNA: Compute them as you see fit. Ovito and ASE already have some of those implemented.
- ML: [Scikit-learn](#), PCA or t-SNE (for dimensionality reduction, if necessary), k-means or DBSCAN for the clustering.
- Visualization: matplotlib, Ovito, ase.view, VMD, POV-Ray,... Be creative!

Data Provided:

- MD trajectories and structures with phase transitions (solid-solid and solid-liquid) or interfaces (nucleation).
<https://drive.google.com/drive/folders/1zOOWD7RAo1ipuf1JA7EpVUxM1MvjRMPq?usp=sharing>

Goals:

- Extract meaningful descriptors of atomic environments
- Apply clustering or classification to label phases or intermediates
- Compare with coordination/CNA analysis in clarity and interpretability
- Generate 2D plots or animations of structural evolution

Project 2: Benchmarking DFT Methods for Small Hydrocarbons *[Workflow, DFT]*

Reference: João Paulo Almeida de Mendonça

Objective: Evaluate the performance of different Density Functional Theory (DFT) methods in computing the properties of small organic molecules. Identify systematic trends in energies and electronic properties across functionals.

Background: Different DFT functionals (LDA, PBE, B3LYP, etc.) yield varying results, and their reliability can depend on molecular type and on the property that you're trying to predict. By benchmarking key properties such as atomization energy, HOMO-LUMO gaps, and Mulliken charges, we can understand how consistent different methods are and identify potential trade-offs between accuracy and computational cost.

Tools & Libraries:

- Python + ASE + [PySCF](#): This combo will allow you to run all these simulations in a notebook. It isn't the most computationally effective for production, but is very practical for implementation! We recommend this one for the practice here.
- Bash + [Quantum Espresso](#)/[ORCA](#)/[GAMESS](#)... : This is the standard combo people use in the HPC facilities. If you have experience on this approach, be free to go for it!
- Professional workflows using tools like [AiiDA](#)/[PyIRON](#)/ASE workflows... are the state of the art for this kind of application, but can be challenging for a few hours practice. We are listing it here just for your information.
- Matplotlib and Seaborn for data analysis and plotting! Be free also to visualize the molecules if you wish to do so.

Molecule Set: Use a filtered version of [ase.collections.g2](#), keeping only molecules composed of C, H, O, and N atoms. ~10–20 molecules recommended.

Functionals to Compare: LDA, PBE, PBE0, B3LYP... If you already use a DFT approach in your work, you can add it here!

Properties to Analyze: Atomization energy, HOMO-LUMO gap, Mulliken charges, computation time and number of SCF steps (as a convergence/cost metric). Feel free to explore the outputs of your calculations and use any other property that you think are interesting.

Questions to Explore

- Do atomization energies correlate well across functionals?
- Electronic properties like HOMO-LUMO gaps are functional dependent in the same way?
- Do SCF convergence steps suggest some molecules are inherently harder to solve?
- Can we say the same for functionals? I.e., are some functionals harder to converge than others?

Project 3: Clustering and Visualizing Relationships Among 200 DFT Functionals [*Unsupervised Learning, Data visualization*]

Reference: João Paulo Almeida de Mendonça

Objective: You will explore how different density functional theory (DFT) methods relate to each other using the extensive dataset of ~200 functionals evaluated over an extensive dataset combination for main group chemistry properties (MGCDB84). Use clustering to detect families of related functionals and embed them in 2D for visualization.

Background: In an impressive [review paper](#), Mardirossian & Head-Gordon benchmarked functionals across diverse chemical datasets for properties like non-covalent interactions, thermochemistry, barrier heights, etc. The supplementary data provides per-functional errors on each dataset. This rich, labeled dataset allows an unprecedented comparative analysis across methods.

Tools & Libraries:

- Python, pandas, NumPy
- Scikit-learn (clustering: hierarchical, k-means, DBSCAN)
- Dimensionality reduction: PCA, t-SNE, UMAP
- Visualization: matplotlib, seaborn, or Plotly

Data Provided: <https://www.tandfonline.com/doi/suppl/10.1080/00268976.2017.1333644>.

Goals

- Load and preprocess per-functional RMSD or MAE vectors across all datasets that compose MGCDB84. Apply clustering to try to detect groups of functionals that behave similarly.
- Reduce to 2D embedding to visually map clusters and inspect the result for relationships. Labeling the functionals or coloring the scatter plot in terms of the functional type or MRSD on targeted datasets can help. Try to interpret the clustering physically.

Challenges & Considerations:

- You will have to choose on doing or not some kind of normalization before clustering, since the errors on some quantities have different physical meanings and orders of magnitude.
- Try to optimize the clustering parameters (number of clusters of k-means, density for DBSCAN, etc.) based on some quantitative criteria and examine if this is enough sensitivity to have a clustering that is still physically relevant.
- If the clustering method or the visualization highlights outliers, try to check what makes those cases special. For instance, is this method particularly bad/good for a given property?

Project 4: Coding a DIY MLIP [*Supervised Learning, Molecular Descriptors, Molecular Dynamics*]

Reference: João Paulo Almeida de Mendonça / Irina Piazza

Objective: Your work will be doing by hand what you saw many codes doing this week: Use the MD data from this week (for instance, the one in aluminium) and convert the structures in local environment descriptors, code a neural network regressor and try to learn the energy as function of those local descriptors.

Background: Many codes in the literature already do this training process for us, but there is a value in doing it by hand: Comprehending each detail of the process, including data manipulations and metaparameters choice. Also, learning only the energies make this a problem handable in the time we have this afternoon.

Tools & Libraries:

- Everything can be done in a Notebook, using !pip to install packages.
- Use the notebooks from this week to get the MD data. You can use LAMMPS with a simple potential like EAM or even Lennard Jones.
- Descriptors: Ewald sum matrix, SOAP, Behler-Parrinello/ACSF... Check what you can find in [DScrive](#) or do manually your own feature engineering.
- ML: [Scikit-learn](#) has a multi layer perceptron that can do the trick. Otherwise, check [Torch](#) or [Tensorflow](#).
- Visualization: matplotlib, Ovito, ase.view, VMD, POVRAY,... Be creative!

Goals:

- Generate MD data.
- Build a full MLIP model, including descriptor and neural network.
- Do the training and check the learning curves. At this point, part of the group can work in the next goal while one takes care of testing different hyperparameters.
- Validate the result: Remember to plot the learning curve. You can validate the results by also plotting the predicted vs real energies in the validation set; and the potential energy of two atoms isolated in vacuum as function of the distance between them.

Project 5: MLIP on Aluminium complete database [*Supervised Learning, MLIP, Molecular Dynamics*]

Reference: Irina Piazza

Objective: Discriminate between physical results from ML artefact applied to the Aluminium dataset generated during MLIP hands-on. See [MLIP hands-on link](#)

Background: linux command, python, code, linux/win/mac terminal

Questions:

- 1) Suppose that I want to train my model playing with the epochs parameter to improve the accuracy of my NN model. You have to study the minimum RMSE varying the number of epochs for 10 different values of epochs. Are you doing only one train or 10 different trains? Motivate your answer.
- 2) What are the hidden layers? How can you establish how many node beach layers should have it?

Instructions:

- Open [MLIP hands-on link](#)
- Make a copy of this notebook and rename it for example, *nb_projet_5_MLIP*.
- Run again the cells that allow you to obtain an Aluminum dataset with 200 snapshot.
- Create 3 different MLIP models changing the NN architecture into:
 - Case 1: `global_nodes_short = 2 2` (2 hidden layers with 2 nodes each).
 - Train with epochs = 50,100,200
 - Case 2: `global_nodes_short = 5 5` (2 hidden layers with 5 nodes each).
 - Train with epochs = 50,100,200
 - Case 3: `global_nodes_short = 10 10` (2 hidden layers with 10 nodes each).
 - Train with epochs = 50,100,200
- Make a plot with all minimum RMSE. Could you comment this plot? Can you conclude something?
- Have a look at how best weights and learning-curve.out change and how RDF changes. Have you noticed any artifacts? If yes, in which model? Comment and motivate your answer.

Tools & Libraries: Colab, n2p2, lammmps

Goals:

- To learn how hyperparameters can optimize your NN MLIP model

Project 6: MLIP n2p2 workflow (WF) [*Supervised Learning, MLIP, Molecular Dynamics*]

Reference: Irina Piazza / Akshay Krishna

Objective: build a workflow with data generation, scaling, training and validation.

Background: linux command, python, code, linux/win/mac terminal

Instructions:

- Open [MLIP hands-on link](#)
- Make a copy of this notebook and rename it for example, *nb_projet_6_MLIP*.
- Using the [subprocess python library](#) converts the notebook project into a workflow where you have 4 steps:
 - Data generation
 - Scaling
 - Training
 - validation

Tools & Libraries: python, terminal.

Goals:

- **Build a WF**

Project 7: Apptainer Image to run a Molecular Dynamic / Your experiment [*Containers, Reproducibility, Parallel computing*]

Reference: Léo Orveillon

Objective: To either build a basic LAMMPS image or one for a code you are using for your research. (For LAMMPS) Once the container is ready, we will test it on a simple Molecular Dynamics, we'll quickly see the need for parallelization and try to build a new image with support for that.

Background: Using containers does not only help us get rid of the isolation and portability problem, it also helps with simulation reproducibility, which is needed when doing science. Most software evolves through time, and if one of the libraries you use changes some functionality, the output of one simulation can also change through time. That's why being aware of the software you use and their version is important to avoid this problem, or at least being aware that it could happen.

Tools & Libraries:

- Apptainer
- Open MPI
- Linux

Goals:

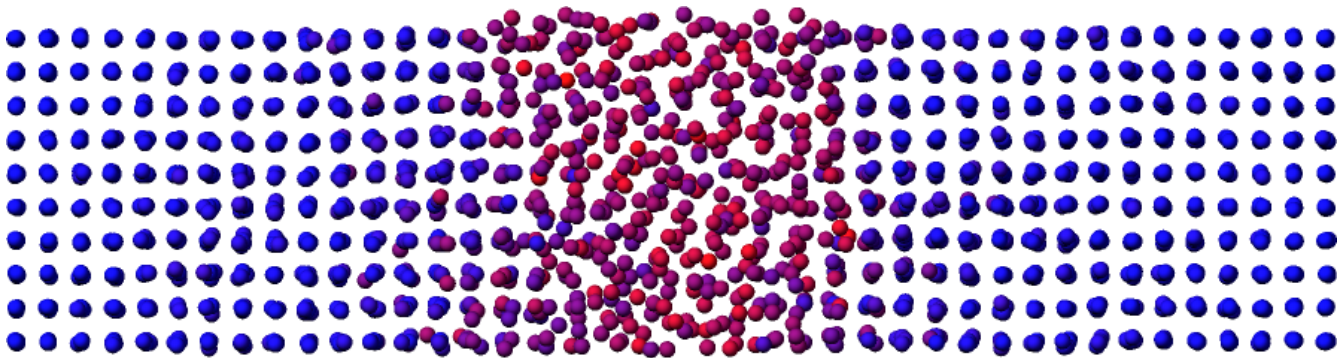
- Having a better understanding of software environment
- Learn how to build a container to share your software/experiment

Project 8: “*God made the bulk; the surface was invented by the devil*”: Can machine learning handle the interface?

Reference: Simon Gravelle, Irina Piazza

Quote: Wolfgang Pauli

Objective: Investigate solid-liquid coexistence in aluminum and compare the ability of an EAM potential and an MLIP to describe the melting temperature and dynamics of a solid-liquid interface.



Tools & Libraries (visit <https://github.com/simongravelle/DIADEM> or click on the links below):

- [LAMMPS GUI](#)
- Provided [starting input file](#) (a cubic FCC crystal)
- Provided [EAM potential](#)
- Provided [MLIP potential](#) (created during Wednesday morning session by Irina Piazza)

Instructions:

- Start from the provided cubic FCC crystal and modify the box dimension to elongate it.
- Create a solid-liquid coexistence configuration using two thermostats: a hot region to melt part of the crystal and a cold region to maintain the solid phase.
- Visualize the solid and liquid phases using the [centrosymmetry parameter](#).
- Turn off the two thermostats, and replace them with a single thermostat. Observe whether the crystal grows, shrinks, or the two phases coexist.
- Replace the EAM potential with the MLIP potential and repeat the same experiment.
- Compare the interface evolution and discuss whether the MLIP reproduces the same behavior as the EAM potential.

Question: Does the MLIP potential, which was learned from bulk phases rather than from interfaces, still reproduce the solid-liquid coexistence behavior of the EAM potential ?